



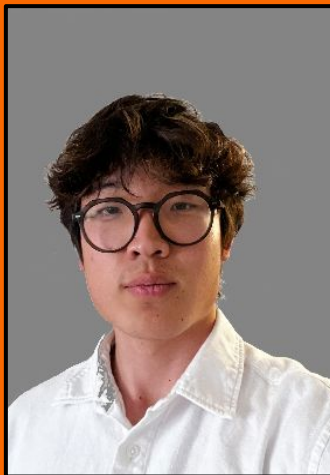
RIT

Faculty Project #4: Open-source malware sandboxes analysis

By:

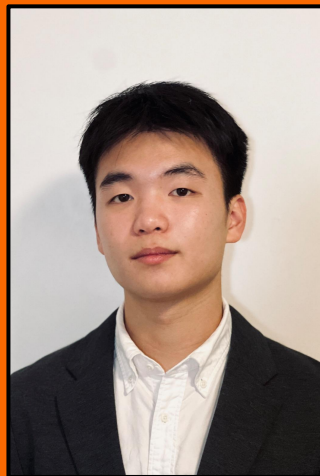
Donovan, Han, Hayden, Roberto

Meet Our Team



Donovan Hwang

Degree:
Masters in
Cybersecurity



Han Chen

Degree:
Bachelor in
Cybersecurity



Hayden Orr

Degree:
Bachelor in
Cybersecurity

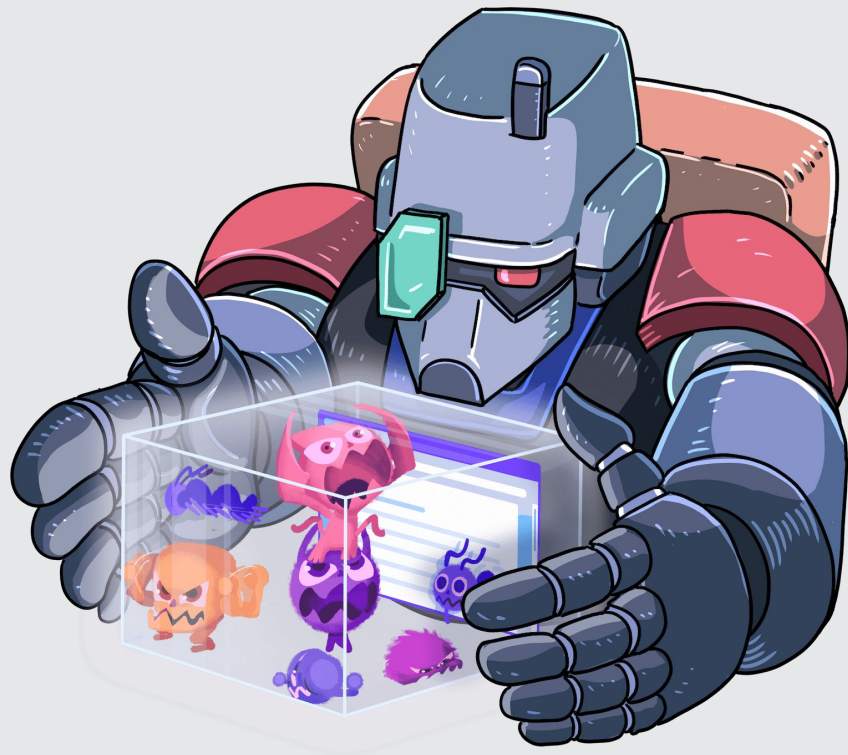


Roberto Reade

Degree:
Masters in
Cybersecurity

What is a Malware Sandbox?

- Controlled environment where suspicious files are executed safely
- Observe behavior instead of just code
- Outputs:
 - Process activity
 - Network communication
 - File/registry changes
- Why it matters:
 - Modern malware hides until execution



Project Motivation

The Old Paradigm

Decline of Disruption

- 38% relative decrease in overt ransomware encryption.



The Modern Threat

- Cryptographic wrappers and runtime packers conceal true intent until execution in memory
- ANY.RUN reports 6.8 million automated sandbox sessions annually
- ***Manual reverse engineering cannot scale***

Project Description

Task

- Identify state-of-the-art sandboxes employed in dynamic malware analysis
- Configure and deploy selected environments
- Execute malware samples for testing
- Compare sandboxes outputs

Scope

3 Sandboxes:

CAPE, Cuckoo3, and DRAKVUF

Malware Samples:

Two samples from each of the 6 malware families

Control Group:

Two benign files for comparison

The Limits:

Only test Windows malware, ignoring Mac or Linux

Malware Selection

Control (Benign)

- 7na
- Process Explorer

Ransom

- HiveRansomware
- WannaCry

Worm

- EternalRock
- RaspberryRobin

Botnet

- QakBot
- TrickBot

Trojan

- AsyncRAT
- Remcos

Spyware / Infostealer

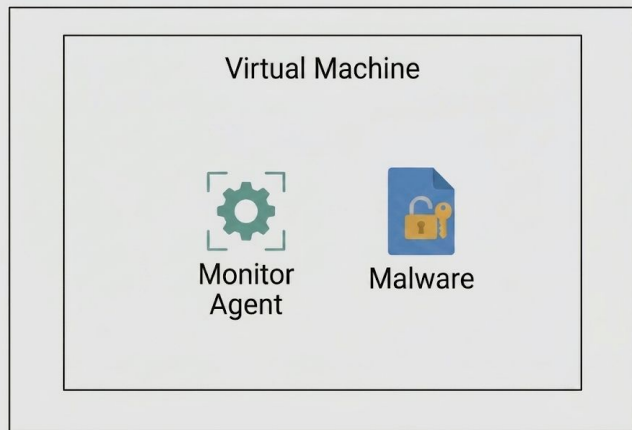
- RedLine
- LummaStealer

Rootkit

- BlackLotus Rootkit
- RustyClaw

Types of Monitoring

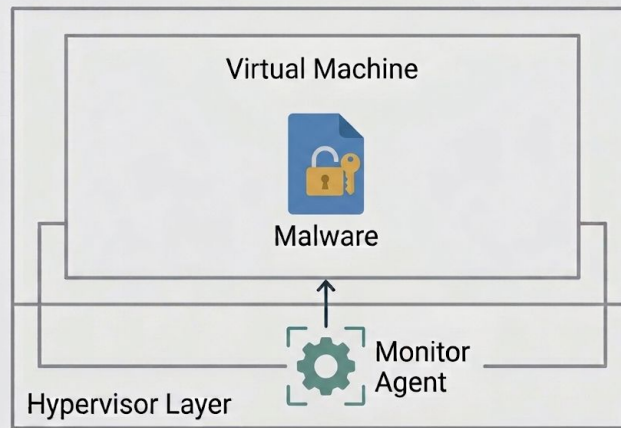
Inside-Guest



Platform: CAPE / Cuckoo3

- Uses in-guest Python agents (Ring 3) or API hooking.
- Tradeoff: High semantic context, but leaves visible footprints for evasive malware to detect.

Outside-Guest



Platform: DRAKVUF

- Monitors via Xen-based VMI.
- Observes low-level OS and kernel behavior.
- Tradeoff: Limited to only Intel CPU with VT-x and Extended Page Tables (EPT); Exceptional stealth, but captures massive system-wide noise.

Methodology - Setup

Cuckoo3

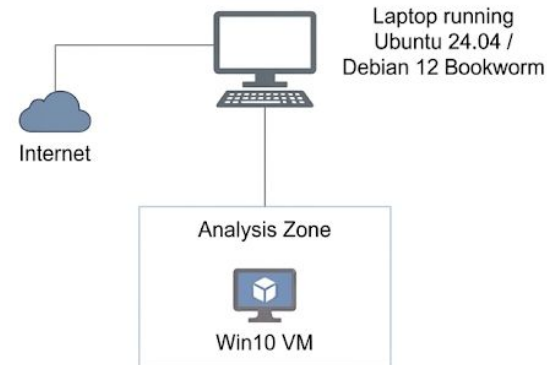
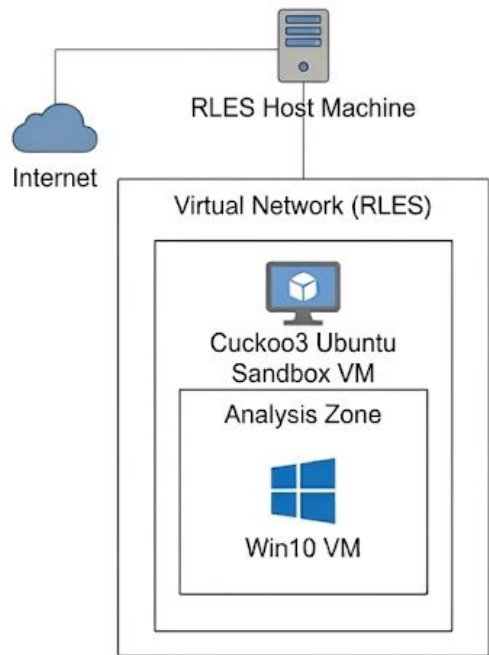
Cloud-based nested virtualization (RLES Virtual Machine - Ubuntu 22.04).

CAPE Sandbox

Non-distributed physical host via VMware (Ubuntu 24.04).

DRAKVUF

Non-distributed dedicated physical host via Xen hypervisor (Debian 12.13).



Forensic Visibility Compared

- **Baseline:** All platform natively capture process trees, PCAPs, connections, and screenshots.
- **CAPE:** Broadest direct coverage; excels in automated payload extraction and memory debugging.
- **DRAKVUF:** Unmatched low-level tracing; captures system-wide OS events without in-guest agents.
- **Cuckoo3:** Solid baseline, but relies heavily on indirect or raw logs for deeper metrics.

✓	×	△
Yes	No	Indirect, raw-log only

Artifact type	CAPE	Cuckoo3	DRAKVUF
Processes / process tree	✓	✓	✓
Network connections	✓	✓	✓
DNS / domains	✓	✓	✓
HTTP requests / URLs	✓	×	✓
Raw PCAP file	✓	✓	✓
File activity	✓	△	✓
Registry activity	✓	△	✓
Per-process API calls	✓	×	✓
System-wide syscall / OS event trace	×	×	✓
Memory dump artifacts	✓	×	✓
Screenshots	✓	✓	✓
Services / startup persistence	✓	△	△
Mutexes	✓	△	△
Executed commands	✓	△	△
Extracted payloads / configs	✓	×	×
TLS session keys	✓	×	✓

Sandbox Scoring

- CAPE and Cuckoo3 showed different score calibration
- CAPE labeled 7za clean; both flagged Process Explorer
- Cuckoo3 scored BlackLotus and QakBot higher than CAPE
- CAPE provides malware family matching
- Family matching was limited: 4/12 matches, 2 exact

sample_name	severity_score/ malstatus	severity_score	detected_family
7za	0.5/Clean	4	AsyncRAT
AsyncRAT	10/Malicious	10	
BlackLotus	1.7/Clean	7	
EternalRock	9/Malicious	4	
HiveRansom	2.7/Clean	4	
LummaStealer	9/Malicious	8	
QakBot	2/Clean	9	
RaspberryRobin	8/Malicious	7	Arechclient2
Redline	10/Malicious	7	
Remcos	10/Malicious	7	
RustyClaw	8/Malicious	7	HiddenVNC
TrickBot	10/Malicious	7	
procexp64	7/Malicious	9	
wannacry	8/Malicious	8	

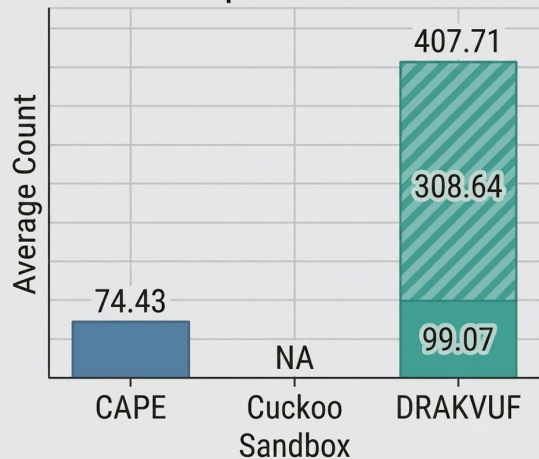
Color Key

Cape

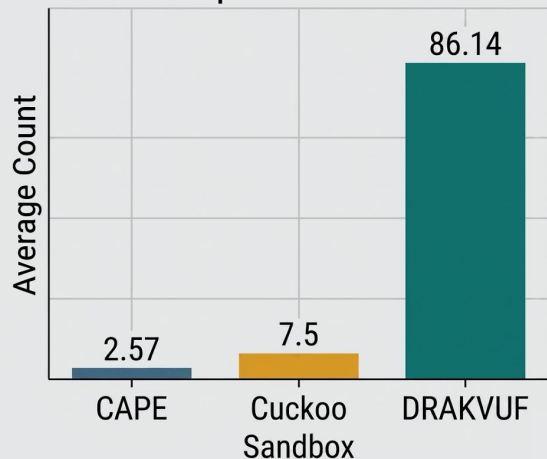
Cuckoo

Some Metrics:

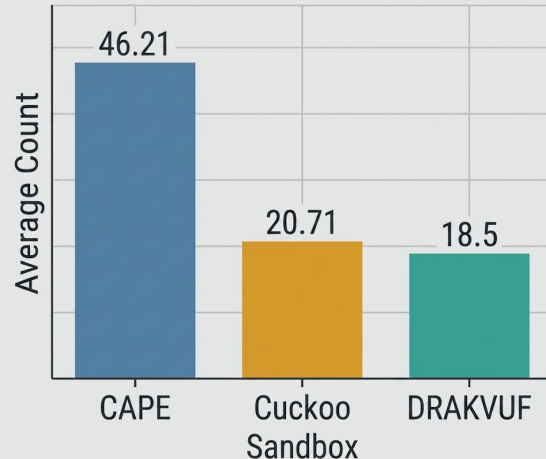
Unique API Calls



Unique Processes



Network Connections



Selecting the Right Lens

CAPE Sandbox

Best For:

- Deep payload extraction
- Memory dumping
- Dynamic unpacking

Strength:

- Provides automated YARA categorization
- Actionable Indicators of Compromise (IoCs) upon deployment

DRAKVUF

Best For:

- Analyze evasion-aware malware and rootkits that require stealth

Strength:

- Offers unmatched kernel-level visibility
 - Invisible, agentless hypervisor design

Cuckoo3

Best For:

- Serves as an accessible modular starting point
 - For basic behavioral triage and general analysis

Strength:

- Maintains relatively lightweight operational overhead

Constraints and Context

Default Configuration

Platforms were evaluated largely under baseline setups.

Extensibility via custom plugins and complex `.conf` field remains a deeper area to explore.

OS Constraints

Restricted entirely to Windows 10 victim environments to maintain experimental control.

Linux and MacOS detonation behaviors were unexamined due to time and resources constraints.

Capability vs. Fidelity

This study measured the presence of generated artifacts (what the tool outputs), not strictly their absolute accuracy against a reverse-engineered ground truth.

Questions?

Challenges - Cuckoo3

Installation Dependency

Strict order is required for a functional environment:

- Dependencies → VMCloak → Cuckoo3
- Deviations cause unstable or partial configurations

Web Interface Issues

Challenges with the GUI access and security:

- CSRF protection errors blocked access initially
- Required manual `CSRF_TRUSTED_ORIGINS` config

Capture Limitations

Restrictions on network packet capture:

- tcpdump lacks default security permissions
- Required AppArmor bypass to enable capture

Challenges - CAPE

OS Compatibility

- Ubuntu 22.04 → repeated failures (KVM/QEMU, dependencies)
- Migrate to Ubuntu 24.04 for better stability.

Configuration

- “dnsmasq.service” failed during setup
- Modify systemd-resolved setting to remove conflict.

Permissions

- VM creation failed due to ISO access restrictions
- Fixed by moving ISO → /var/lib/libvirt/images/

Challenges - DRAKVUF

Virtualization

- **Incompatibility:** Initial cloud deployment (RLES) failed despite VT-x.
- Attempted VMware; still failed despite VT-x.
- Moved to dedicated bare-metal (Debian 12).

Xen Build & Deps

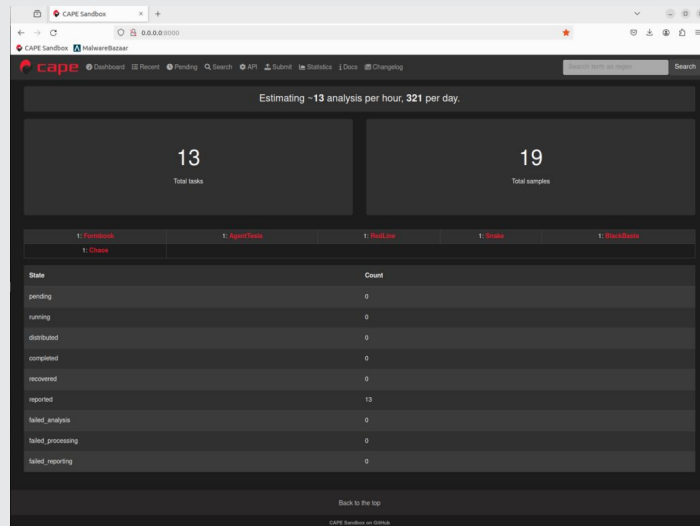
- **Firmware:** Broken OVMF reference; required manual EDK2 patch to a stable GitHub revision of EDK2 (edk2-stable202408.01).
- **Caching:** Submodule issues caused repeated build failures.

System & Boot

- **Environment:** Incorrect PATH missing `ldconfig` & `update-grub`.
- **GRUB Failure:** Xen not detected; manual fix in `/etc/grub.d/20_linux_xen` by defining more variables.

CAPE: Malware Analysis

- Executes suspicious files inside an isolated virtual machine
- API hooking to monitor malware behavior in real time
- Logs file creation and deletion activity
- Captures network traffic, screenshots, and full memory dumps
- Generates detailed reports on tasks, file changes, and C2 communications
- Unpacks malware and categorizes it using YARA signatures
- Extracts malware configuration data
- Uses a custom debugger to bypass evasion techniques



Cuckoo3: Malware Analysis

-

DRAKVUK: Malware Analysis

- Executes inside an isolated Xen-based virtual machine
- Uses Virtual Machine Introspection (VMI) instead of in-guest API hooking
- Produces detailed behavioral logs and forensic traces for analysis
- Detects unpacking behavior and hidden execution flows through low-level tracing
- Extracts indicators of compromise (IOCs) such as file paths, domains, and memory artifacts
- Resistant to anti-analysis and evasion techniques due to agentless design

The screenshot displays the DRAKVUK web interface. On the left is a dark sidebar with navigation links: ANALYSIS, Upload sample, API docs, and RQ Dashboard. The main content area is titled 'Analysis report' and shows a process tree for 'Listdlls64.exe'. The tree includes processes like System, csrss.exe, wininit.exe, winlogon.exe, fontdrvhost.exe, dwm.exe, userinit.exe, explorer.exe, SecurityHealthSystray.exe, Listdlls64.exe, and MushNotificIcon.exe. To the right of the tree is a table with job details: File name (Listdlls64.exe), SHA256, Type (PE32+ executable), Start command, Analysis time (600 seconds), Job started at, Execution started at, and Job finished at. Below the table are buttons for 'Download PCAP', 'T15 keys', and 'Memory dump'. At the bottom, there are tabs for 'Summary report', 'General logs', 'Process info', and 'Screenshots', with 'Process creation' selected and showing a 'Success' status.

File name	SHA256	Type	Start command	Analysis time	Job started at	Execution started at	Job finished at
Listdlls64.exe	29d23bc492e48a5ae6844302d3430e07d08e0427b53aa70d93	PE32+ executable (console) x86-64, for MS Windows, 6 sections	C:\Users\zen\Desktop\Listdlls64.exe	600 seconds	2025-07-23T09:57:53.615812+00:00	2025-07-23T09:59:01.731864+00:00	2025-07-23T10:09:50.129064+00:00

Plugins: apimon, ftrace, memdump, procmon, socketmon, tmon

Download PCAP T15 keys Memory dump

Summary report General logs Process info Screenshots

Process creation Success